

Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

Extreme Programming

Período: **08/05/2026** a **03/07/2026**

Em grupo | Todos os grupos abordam os 5 temas | 8 sprints + entrega final

XP em conjunto com Scrum: XP + Scrum híbrido

Sprint 6 — Integração com o Ecossistema Ágil (19/06/2026) • Mapeamento de cerimônias e práticas complementares

Scrum e Extreme Programming (XP) atuam em níveis diferentes e, por isso, se completam. **Scrum** é um framework de **gestão**: organiza o trabalho em sprints, define papéis e cerimônias e cuida do “o quê” e do “quando” entregar. Já o **XP** é um conjunto de práticas de **engenharia**: TDD, programação em par, integração contínua e refatoração cuidam do “como” construir software com qualidade. Como o Scrum não prescreve práticas técnicas, combiná-lo com o XP preenche essa lacuna — é exatamente o que fazemos no projeto **Scrunic**.

1. Por que combinar XP e Scrum

Scrum traz ritmo, previsibilidade e organização do fluxo de entrega, mas é silencioso quanto à forma de escrever código. Sem disciplina técnica, um time Scrum pode entregar incrementos a cada sprint e, ainda assim, acumular dívida técnica e defeitos. O XP fornece justamente as práticas que sustentam a qualidade ao longo dos sprints. A tabela abaixo mostra como as camadas se complementam.

Camada	Scrum responde	XP responde
Propósito	Gestão do trabalho e do produto.	Engenharia e qualidade do código.
Pergunta	O quê e quando entregar?	Como construir com qualidade?
Foco	Sprints, papéis, priorização.	TDD, par, integração contínua, refatoração.
Risco se usado sozinho	Dívida técnica acumulada.	Falta de ritmo e de visão de produto.

2. Mapeamento de cerimônias × práticas XP

Cada cerimônia do Scrum se torna mais eficaz quando combinada com práticas de engenharia do XP. O quadro a seguir mapeia, cerimônia por cerimônia, como as práticas do XP se encaixam no nosso fluxo.

Cerimônia Scrum	Objetivo no Scrum	Prática XP complementar	Como se integra no Scrumic
Sprint Planning	Selecionar e estimar o que será feito no sprint.	Histórias + planejamento orientado a testes	As histórias já entram com critérios de aceite que viram casos de teste (TDD) na sequência.
Daily Scrum	Sincronizar a equipe e remover impedimentos.	Pair Programming e propriedade coletiva	Definimos os pares do dia e quem revisa o quê; impedimentos técnicos viram pauta imediata.
Desenvolvimento (durante o Sprint)	Construir o incremento.	TDD, Integração Contínua, Refatoração	Cada commit dispara a suíte de testes; o código é refatorado continuamente para manter a simplicidade.
Sprint Review	Apresentar o incremento aos stakeholders.	Pequenas releases	O incremento revisado já está integrado e testado, pronto para liberar em pequenas versões.
Sprint Retrospective	Melhorar o processo da equipe.	Feedback e coragem para refatorar	Métricas dos testes e da integração contínua orientam melhorias técnicas no próximo sprint.

Ideia central: o Scrum dá o ritmo (sprints e cerimônias) e o XP garante que cada incremento produzido nesse ritmo seja de alta qualidade técnica. Um cuida do compasso, o outro da execução.

3. Práticas XP aplicadas no nosso ciclo Scrum

TDD (Test-Driven Development)

Durante o sprint, cada história do backlog vira teste antes de virar código. No módulo de **Criação de Tarefas** do Scrumic, escrevemos primeiro o teste `deveRejeitarTarefaSemTitulo()` (fase vermelha), depois o código mínimo para ele passar (fase verde) e, por fim, refatoramos. Assim, a meta do sprint nasce já protegida por testes automatizados.

```
@Test
public void deveRejeitarTarefaSemTitulo() {
    Tarefa tarefa = new Tarefa("");
    assertFalse(tarefa.isValida());
}
```

Pair Programming (Programação em Par)

As tarefas mais complexas do sprint — como o quadro Kanban interativo em React/TypeScript, com arrastar-e-soltar dos cartões — são feitas em par. Um desenvolvedor atua como “piloto”, escrevendo o código, e o outro como “navegador”, revisando em tempo real. Isso reduz retrabalho e antecipa problemas ainda dentro do sprint, alimentando diretamente a Daily com decisões técnicas.

Integração Contínua

Com vários desenvolvedores atuando ao mesmo tempo nos módulos de login, cadastro e gestão de usuários, a integração contínua garante que o incremento do sprint permaneça estável. A cada commit, a aplicação (Node.js + MySQL) é compilada e toda a suíte de testes do TDD é executada automaticamente; se algo quebra, o envio é rejeitado na hora. Isso mantém o incremento sempre pronto para a Sprint Review.

4. Fluxo de uma sprint híbrida XP + Scrum

Um sprint típico do Scrunic combina as duas abordagens da seguinte forma:

- **Planning (Scrum):** a equipe seleciona histórias e define a meta do sprint.
- **Escrita de testes (XP – TDD):** os critérios de aceite viram testes automatizados.
- **Implementação em par (XP – Pair Programming):** piloto e navegador desenvolvem juntos.
- **Commit + Integração Contínua (XP):** os testes rodam automaticamente a cada envio.
- **Daily (Scrum):** sincronização e ajuste dos pares e prioridades.
- **Refatoração contínua (XP):** o código é mantido simples ao longo de todo o sprint.
- **Review e Retrospective (Scrum):** o incremento testado é apresentado e o processo é aprimorado.

Resultado: a combinação XP + Scrum entrega o melhor dos dois mundos — a previsibilidade e a organização do Scrum somadas à qualidade técnica e à capacidade de absorver mudanças do XP. É um híbrido natural: o Scrum organiza o trabalho e o XP garante que o código produzido em cada sprint seja confiável e fácil de evoluir.

5. Síntese

Elemento	Vem do Scrum	Vem do XP
Organização do tempo	Sprints e cerimônias	—
Papéis e priorização	PO, Scrum Master, Time	—
Qualidade do código	—	TDD e refatoração
Colaboração técnica	—	Pair Programming
Estabilidade do incremento	—	Integração contínua
Entrega de valor	Incremento por sprint	Pequenas releases

Mais do que alternativas, Scrum e XP são camadas complementares: juntas, formam um modelo ágil completo, do planejamento à engenharia.